

Stage 1 Project Design Report - Idrak Islam (219284330)

Project Overview | Real Estate Agent

Problem and Motivation

Property search tools force buyers to think in filters. Minimum price, maximum price, bedroom count, property type. But buyers don't think that way. They think "somewhere quiet where my kids can walk to school, and I don't want to be house-poor." Translating that into filter widgets loses most of the meaning, and the things that matter most, like whether a home is actually affordable with a specific down payment, usually live on a different page.

So buyers end up doing the synthesis themselves. A dozen open tabs, a spreadsheet, and manual reasoning about whether a cheaper fixer-upper really beats a pricier move-in-ready place. The data is all there but what's missing is a system that understands intent and reasons across it.

This project is a conversational real estate advisory agent. The user describes what they want in plain language. The system extracts structured criteria from that, runs deterministic search and financial calculations, and uses an LLM to explain results, surface trade-offs, and point out when the user's requirements can't all be satisfied at once.

The central design commitment is the split between judgment and computation. The language model interprets and explains. All filtering, ranking, and mortgage math is done by ordinary code against a known dataset. This keeps financial figures verifiable, makes every recommended property traceable to a real record, and produces a system that can be tested.

Target Users

Primary: the prospective home buyer. A non-expert, usually buying for the first time. They know what they want in lifestyle terms but not in filter terms. They need help turning preferences into criteria, understanding what they can afford, and comparing shortlisted options. Most use cases are driven by this actor.

Secondary: the real estate agent. A professional assembling options for a client. Same system, different emphasis: fast shortlist building, side-by-side comparison, and generated summaries they can share. Higher listing volume, heavier use of the command-line interface for repeat queries.

Supporting actors. The LLM service, called for requirement extraction and explanation. The local listing data source, which supplies the property records.

Agent Description

The agent advises. It doesn't act. It never transacts, contacts sellers, or does anything with external consequence. Its job is to understand, retrieve, compute, and explain.

Capabilities

Requirement interpretation. Turns natural language into a structured SearchCriteria object. Users give requirements in pieces and sometimes contradict themselves, so criteria accumulate and merge across turns rather than being rebuilt from each message.

Tool selection. The agent holds a set of tools and has to work out which ones a request needs. A question about monthly cost needs the mortgage tool. A question about neighborhood character needs a different one. Choosing correctly, and not calling anything unnecessary, is behavior of the agent rather than a fixed code path, which is why it gets tested behaviorally in Stage 3.

Reasoning and explanation. Given results the deterministic layer produced, the agent explains why certain options fit the user's priorities, what each one gives up, and where the stated requirements conflict.

Memory. Preferences persist across a conversation. A budget given in the first message still constrains the tenth search without being repeated.

Tools available to the agent

Tool	Function
SearchTool	Queries the listing repository using structured criteria
MortgageTool	Computes monthly payment, total interest, and affordability verdict
ComparisonTool	Builds an attribute-by-attribute comparison matrix

Limits on agent authority

The agent can only reference properties the repository handed it. It cannot invent a listing, change a price, or calculate a payment itself. Those are delegated to deterministic components and come back as values the agent may describe but never produce. The architecture enforces this, and behavioral tests in Stage 3 verify it.

AI / LLM Models

The system uses **Claude Sonnet** (Anthropic) through the Messages API. It was chosen mainly for reliable structured output, which matters because requirement extraction has to return schema-conformant JSON that deterministic code consumes directly.

The model is reached only through an LLMClient interface, with two implementations:

- AnthropicLLMClient, which calls the live API.
- MockLLMClient, a deterministic stub returning fixed responses, used throughout the unit tests.

This supports three things. The provider can be swapped without touching application logic. Unit tests run with no network and no API cost. And the boundary between deterministic and AI behavior shows up in the type system instead of living in someone's head.

There are two prompting modes. **Extraction** asks for constrained JSON against a fixed schema at low temperature. **Explanation** asks for prose grounded in a supplied result set, with the prompt restricting the model to the properties it was given.

Overall Architecture

Four layers, with dependencies pointing inward toward the domain.

Presentation layer. A Swing GUI and a command-line interface, both required by the project specification. Neither holds business logic. Both are thin adapters over the service layer. This is what forces real separation of concerns: any logic dropped into a GUI event handler would have to be duplicated for the CLI, so the mistake becomes obvious immediately.

Service layer. RealEstateService, a Facade giving both frontends one coherent API. It coordinates between the agent and the domain and implements no domain logic of its own.

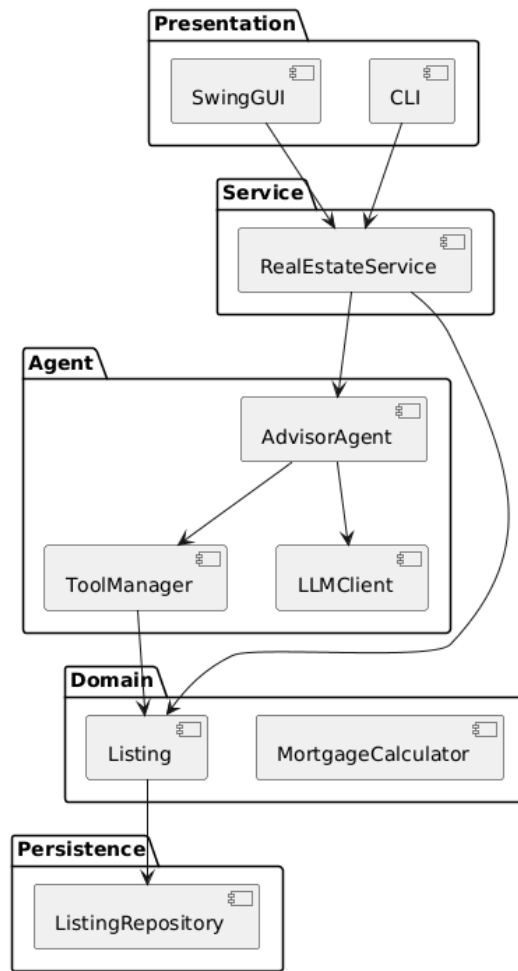
Agent layer. All LLM interaction lives here. RequirementExtractor converts language to criteria, ToolManager dispatches tool calls, MemoryManager holds conversational state, and LLMClient abstracts the provider. Nothing outside this layer talks to a language model.

Domain layer. The entities and the deterministic logic over them: Listing, SearchCriteria, Shortlist, MortgageCalculator, RankingStrategy. This layer doesn't depend on the agent layer at all, which is exactly what lets it be tested exhaustively on its own.

Persistence layer. ListingRepository, in memory, loaded from local JSON at startup. PreferenceStore, which keeps the shortlist and financial profile between sessions. No database, no network.

Dependency direction. Presentation depends on Service. Service depends on Agent and Domain. Agent depends on Domain. Domain depends on nothing above it. Persistence sits underneath Domain and is reached through interfaces the domain defines.

A representative request. A natural-language query comes in through the GUI and reaches RealEstateService, which passes it to AdvisorAgent. The agent extracts criteria through the LLM, merges them with remembered preferences, and picks SearchTool. That tool queries ListingRepository, which applies the filter and returns matching Listing objects. The agent receives them and writes an explanation limited to that set. The service returns listings and explanation together, observers fire, and both frontends render the result in their own way.



Detailed Feature Specifications

Summary Table

ID	Feature	Type
F01	Listing data import	Deterministic
F02	Natural-language property search	Hybrid
F03	Structured criteria filtering	Deterministic
F04	Result ranking	Deterministic
F05	Property detail view	Deterministic
F06	Mortgage payment calculation	Deterministic
F07	Affordability analysis	Deterministic
F08	Property comparison	Hybrid

F09	Shortlist management	Deterministic
F10	Trade-off explanation	AI

Feature Specifications

F01: Listing Data Import

Description. Loads property records from a local data file into the in-memory repository at startup, validating each record and rejecting malformed ones.

Inputs. A JSON or CSV file containing property records with ID, price, bedrooms, bathrooms, square footage, type, neighborhood, year built, and features.

Outputs. A populated ListingRepository and a summary of records loaded and rejected.

Classification. Deterministic.

Workflow.

1. ListingParserFactory selects a parser based on file extension.
2. The parser reads each record and constructs a Listing.
3. Each listing is validated: required fields present, price positive, counts non-negative.
4. Valid listings enter the repository. Invalid ones are logged with a reason and skipped.
5. The load summary is reported.

Error and alternative cases.

- File not found: the application reports the expected path and exits rather than starting empty.
- Unsupported extension: the factory throws UnsupportedOperationException naming supported formats.
- Malformed record: skipped with a reason, load continues. One bad row does not abort the import.

GUI access. Runs automatically at startup; the summary appears in the status bar.

F02: Natural-Language Property Search

Description. Accepts a free-text description of what the user wants, extracts structured criteria, and returns matching properties.

Inputs. Free text such as "three bedroom house under 850k". Criteria accumulated from earlier turns.

Outputs. A ranked list of matching listings and an updated SearchCriteria visible in the GUI panel.

Classification. Hybrid. Extraction is AI-driven; filtering and ranking are deterministic.

Workflow.

1. The user enters a query in the chat panel or via CLI ask.
2. RequirementExtractor calls LLMClient with a schema-constrained prompt and receives JSON.

3. Extracted criteria merge into the session SearchCriteria. Later values override earlier ones; unmentioned fields persist.
4. The agent selects SearchTool through ToolManager.
5. SearchTool queries ListingRepository, which evaluates the filter against every listing.
6. Results are ranked and returned. Observers update the GUI and CLI.

Error and alternative cases.

- Extraction returns invalid JSON: retry once with a stricter prompt, then ask the user to rephrase.
- No criteria extractable: the agent asks a clarifying question rather than searching with empty criteria.
- Zero matches: the system reports which criterion is most restrictive and offers to relax it.
- LLM unavailable: the system falls back to structured filtering through the criteria panel.

GUI access. The chat panel on the right. Extracted criteria appear in the left panel as the search runs, showing that interpretation occurred.

F03: Structured Criteria Filtering

Description. Filters listings using explicit criteria set through form controls, with no language model involvement.

Inputs. Price range, bedroom and bathroom minimums, property type, neighborhood, required features.

Outputs. A filtered list of listings and a match count.

Classification. Deterministic.

Workflow.

1. The user adjusts a criteria control, or issues CLI search with flags.
2. The control updates the session SearchCriteria.
3. SearchCriteria builds a composite filter tree, combining predicates with AND and using OR within multi-value fields.
4. ListingRepository evaluates the tree against each listing.
5. Matches are returned and rendered.

Error and alternative cases.

- Minimum price exceeds maximum: the inversion is flagged and the search does not run.
- All criteria cleared: the full listing set returns, not an empty one.

GUI access. The criteria panel on the left. Changes apply immediately with no submit step.

F04: Result Ranking

Description. Orders results according to a selectable strategy.

Inputs. Matching listings, the selected ranking mode, and the active criteria for match-quality ranking.

Outputs. The same listings in ranked order.

Classification. Deterministic.

Workflow.

1. The user selects a mode: price ascending, price descending, value per square foot, or best match to criteria.
2. RealEstateService sets the corresponding RankingStrategy.
3. The strategy scores each listing.
4. Listings sort by score, with a stable secondary sort on ID.

Error and alternative cases.

- Missing square footage under value ranking: sorted last rather than treated as zero.
- Ties: broken by ID so repeated runs give identical order, which matters for test reproducibility.

GUI access. A dropdown above the results list. CLI equivalent is --sort.

F05: Property Detail View

Description. Displays the full record for one property with derived values.

Inputs. A listing ID.

Outputs. The complete record plus price per square foot, property age, and monthly payment at default terms.

Classification. Deterministic.

Workflow.

1. The user selects a result card or issues CLI show <id>.
2. RealEstateService retrieves the listing by ID.
3. Derived values are computed.
4. The centre panel switches to detail mode.

Error and alternative cases.

- Unknown ID: the system reports no such listing rather than rendering an empty view.
- Missing square footage: price per square foot shows as unavailable, not zero or infinity.

GUI access. Click any result card; a back control returns to the list.

F06: Mortgage Payment Calculation

Description. Computes monthly payment, total interest, and total cost for a property given financing terms.

Inputs. Price, down payment, annual interest rate, amortization period.

Outputs. Monthly principal and interest, total interest over the term, total cost, loan-to-value ratio.

Classification. Deterministic.

Workflow.

1. The user opens the mortgage panel or issues CLI mortgage <id>.
2. Unspecified parameters take configured defaults.
3. MortgageCalculator applies the standard amortization formula using the monthly rate.
4. Total interest is total payments minus principal.
5. Results render alongside the price.

Error and alternative cases.

- Down payment at or above price: reports no financing required and returns zero, not a negative principal.
- Zero interest rate: special-cased, since the formula divides by zero. Payment becomes principal over months.
- Negative or non-numeric input: rejected at the input boundary.

GUI access. A mortgage panel in the detail view with editable fields and live recalculation.

F07: Affordability Analysis

Description. Assesses whether a property is affordable given income, existing debt, and down payment.

Inputs. Annual income, monthly debt obligations, available down payment, target property.

Outputs. A verdict of affordable, marginal, or unaffordable; the gross and total debt service ratios; the maximum supportable price.

Classification. Deterministic.

Workflow.

1. The user enters financial details once per session.
2. MortgageCalculator computes the monthly payment.
3. Estimated tax and heating are added for total housing cost.
4. Gross debt service ratio is housing cost over gross monthly income; total adds existing debt.
5. Both are compared against standard thresholds for the verdict.
6. Maximum supportable price is derived by inverting the calculation at the threshold.

Error and alternative cases.

- Income not provided: the feature prompts for input rather than assuming a default.
- Zero income: reported as unaffordable without attempting division.
- Debt alone exceeding thresholds: flagged specifically, since no price resolves it.

GUI access. An affordability tab in the mortgage panel; a financial profile dialog collects inputs.

F08: Property Comparison

Description. Shows two to four shortlisted properties side by side with a generated summary of the trade-offs.

Inputs. Two to four listing IDs.

Outputs. A comparison matrix by attribute, with the leading property marked per row, plus a natural-language summary.

Classification. Hybrid. The matrix is deterministic; the summary is AI-generated.

Workflow.

1. The user selects properties and chooses Compare, or issues CLI `compare <id> <id>`.
2. ComparisonTool builds the matrix across price, size, bedrooms, bathrooms, age, price per square foot, and monthly payment.
3. For each numeric attribute the tool marks the leader according to whether higher or lower is better.
4. The matrix goes to the agent, which summarizes the trade-offs.
5. The centre panel renders matrix and summary.

Error and alternative cases.

- Fewer than two selected: comparison is unavailable.
- More than four: capped, since wider matrices become unreadable.
- Missing attribute: the cell shows unavailable and is excluded from leader determination, not treated as zero.
- LLM unavailable: the matrix still renders; only the summary is omitted.

GUI access. A Compare button on the shortlist panel, enabled at two or more selections.

F09: Shortlist Management

Description. Saves properties to a shortlist, removes them, and supports undo.

Inputs. A listing ID and an action of add or remove.

Outputs. An updated Shortlist and a confirmation.

Classification. Deterministic.

Workflow.

1. The user triggers a shortlist action from a card, the detail view, or the CLI.
2. The action is wrapped in a Command and executed through the invoker.
3. The invoker applies it and pushes it onto the undo stack.
4. Observers update the shortlist panel and count badge.
5. The shortlist persists to the preference store.

Error and alternative cases.

- Adding an already-shortlisted property: a no-op with a notice, not a duplicate.
- Undo on an empty stack: reported as nothing to undo.
- Persistence failure: the in-memory shortlist stays correct and the user is warned the session will not be saved.

GUI access. A save control on each result card and in the detail view; the shortlist panel sits lower left. Undo is on the Edit menu.

F10: Trade-off Explanation

Description. Explains in plain language why particular properties fit the user's priorities and what each one gives up.

Inputs. Listings returned by a search, the active criteria, and conversation history.

Outputs. A natural-language explanation referencing specific properties and attributes.

Classification. AI.

Workflow.

1. A search or comparison returns listings.
2. AdvisorAgent assembles a prompt with the listings, criteria, and relevant history.
3. The prompt restricts the model to the supplied properties only.
4. LLMClient generates the explanation.
5. The response is validated: every listing ID mentioned must exist in the supplied set.
6. The explanation renders in the chat panel.

Error and alternative cases.

- The response references a listing not supplied: rejected and regenerated once, then results render without narrative. This is the main grounding safeguard and is verified by behavioral tests.
- LLM unavailable: results render with no explanation and a notice.
- Empty result set: the agent explains why nothing matched instead of describing properties.

GUI access. Appears automatically in the chat panel after any search or comparison.

UML Class Diagram

Design Patterns Used

Pattern	Problem it solves	Participants
Facade	GUI and CLI both need the same operations without duplicating coordination logic	RealEstateService
Strategy	Ranking must vary at runtime without conditionals spreading through the repository	RankingStrategy and its implementations
Composite	Filters combine arbitrarily: price AND beds AND (condo OR townhouse)	Filter, AndFilter, OrFilter, leaf filters
Command	Shortlist actions need undo	ShortlistCommand, AddToShortlist, RemoveFromShortlist,

		CommandInvoker
Factory Method	Parser choice depends on file format, decided at load time	ListingParserFactory, ListingParser
Observer	Two frontends must react to the same state change independently	SearchObserver, GUI and CLI renderers
Adapter	The LLM provider must be swappable and stubbable in tests	LLMClient, AnthropicLLMClient, MockLLMClient

Diagram 1a: Core and Domain - Architecture spine

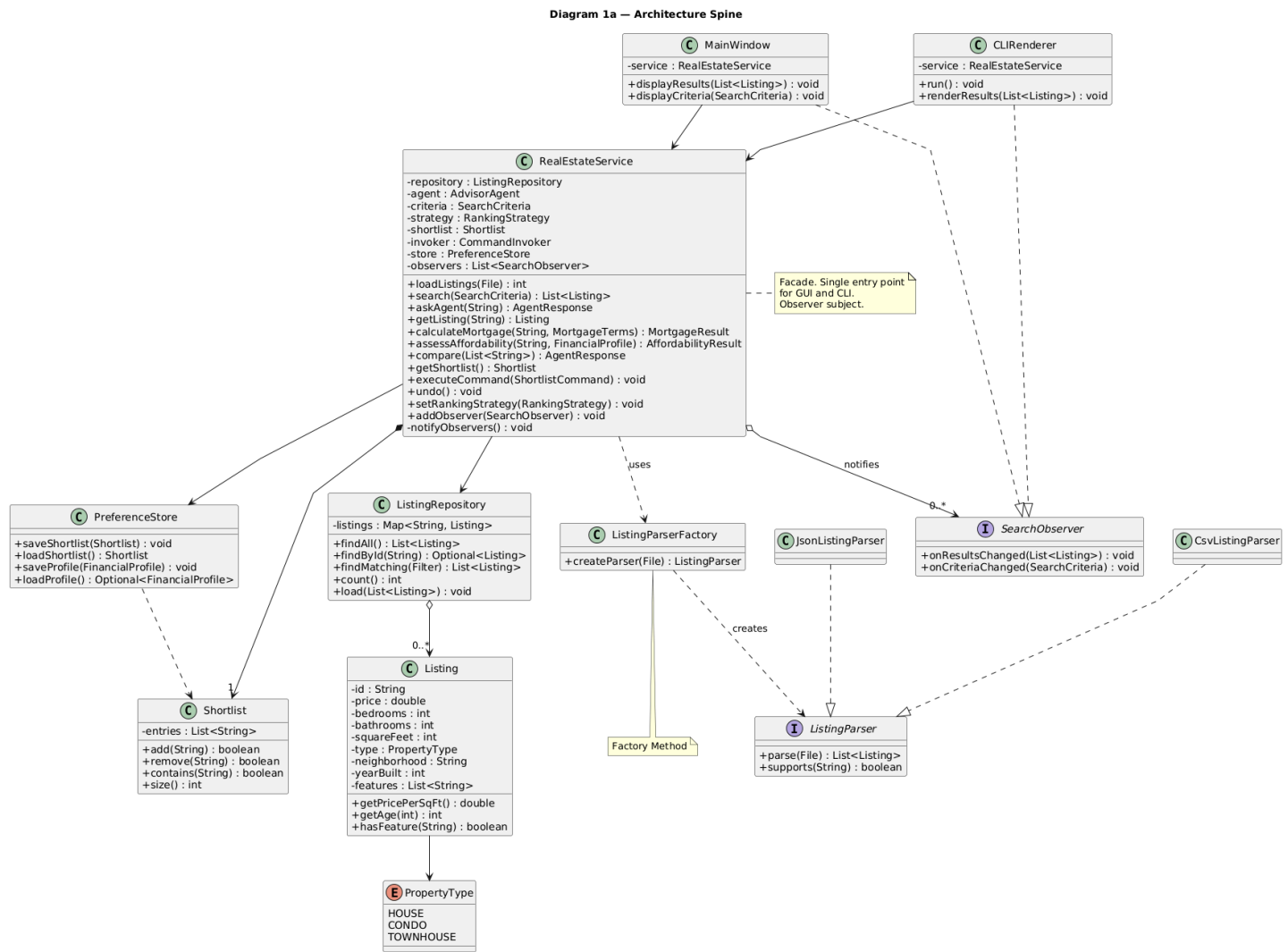


Diagram 1b - Search: Criteria, Filters, Ranking

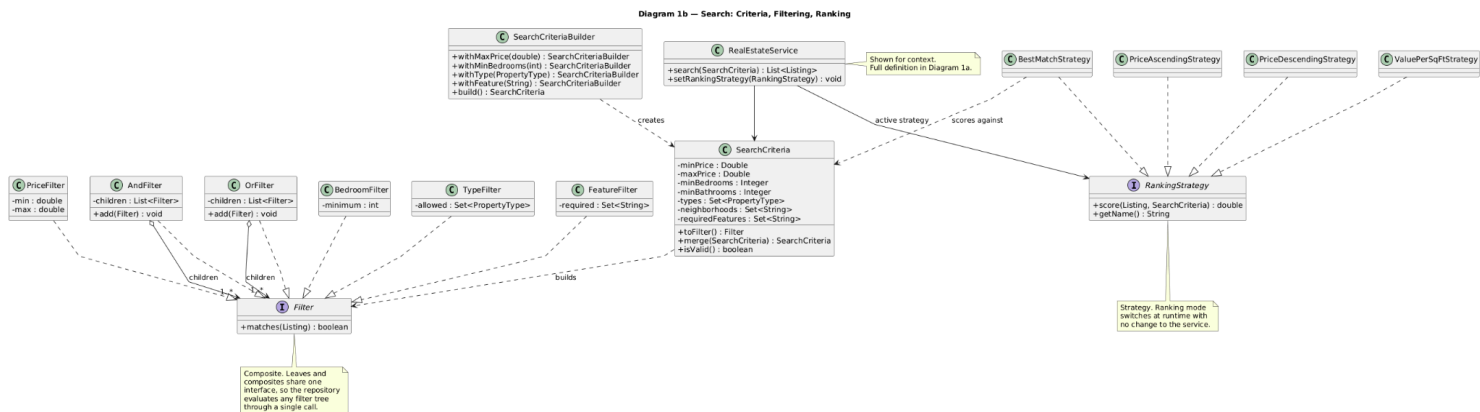


Diagram 1c -Commands and Financial Calculation

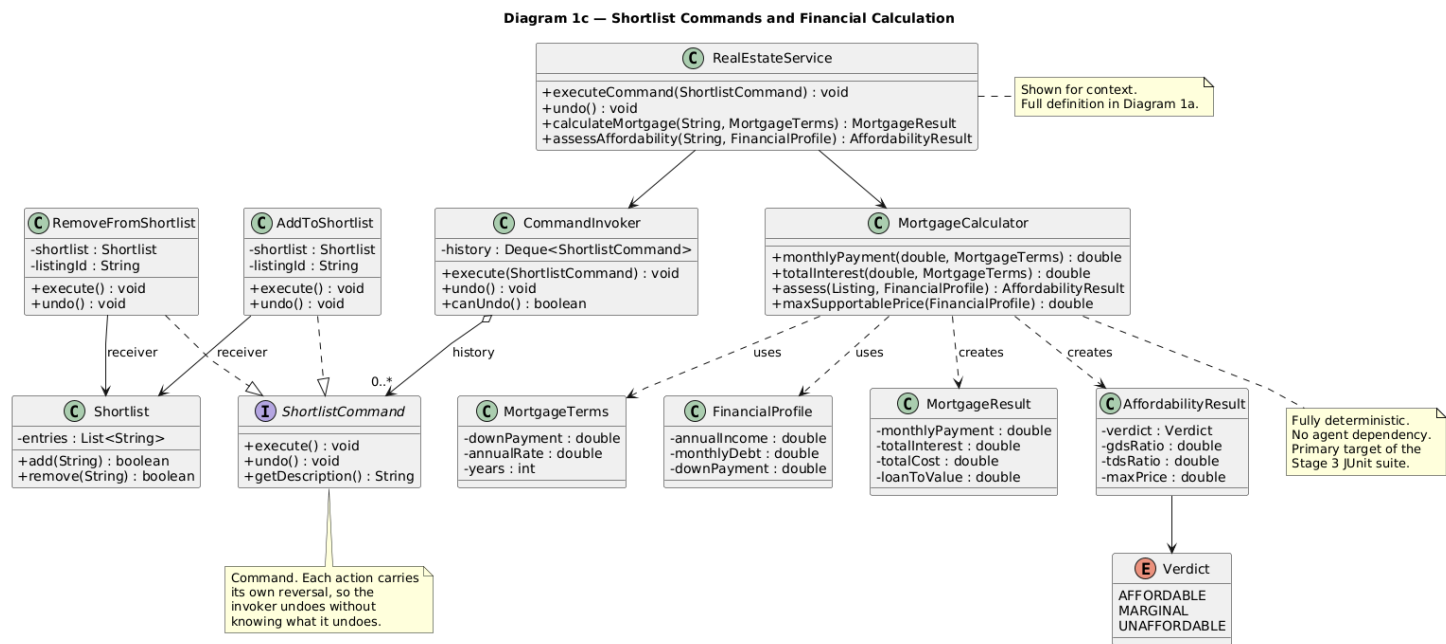


Diagram 2a : Agent Core

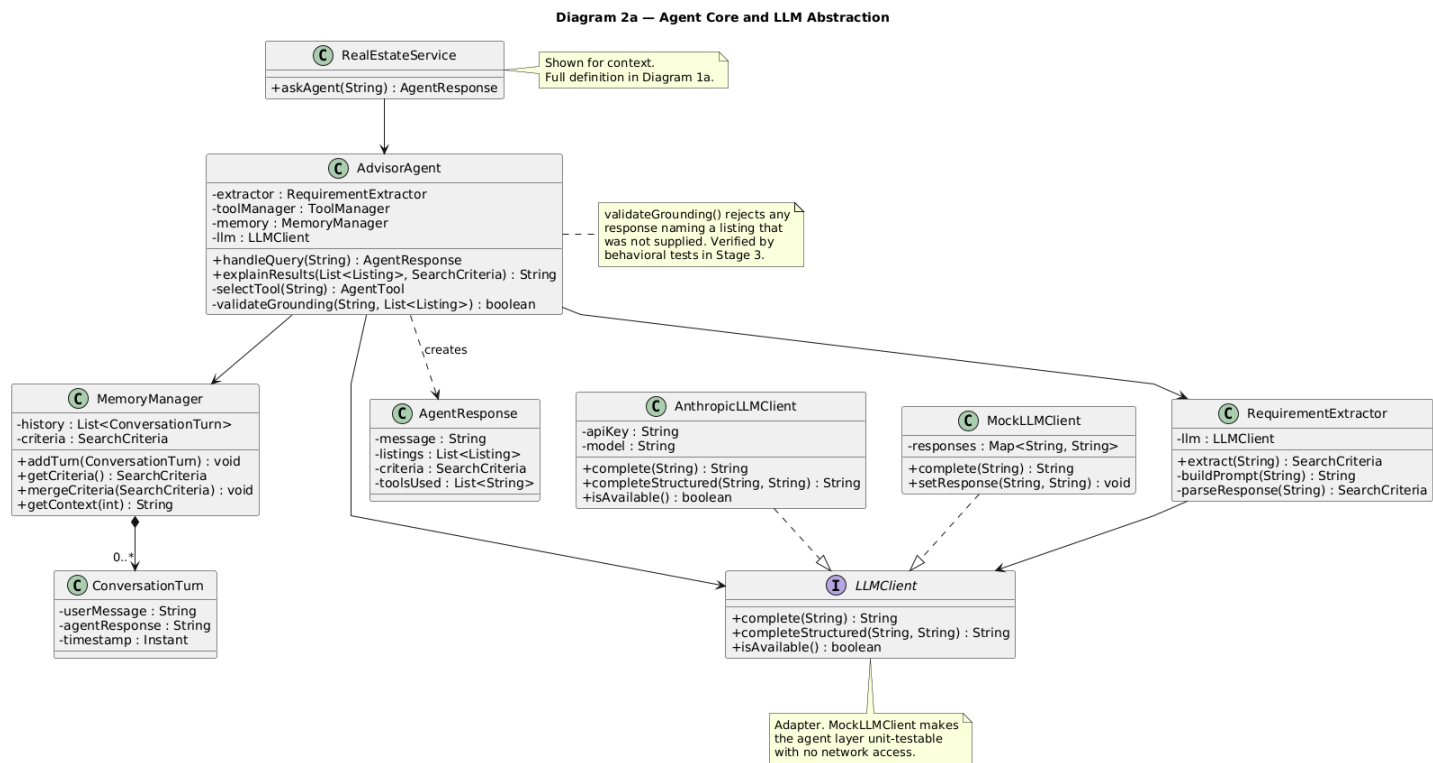
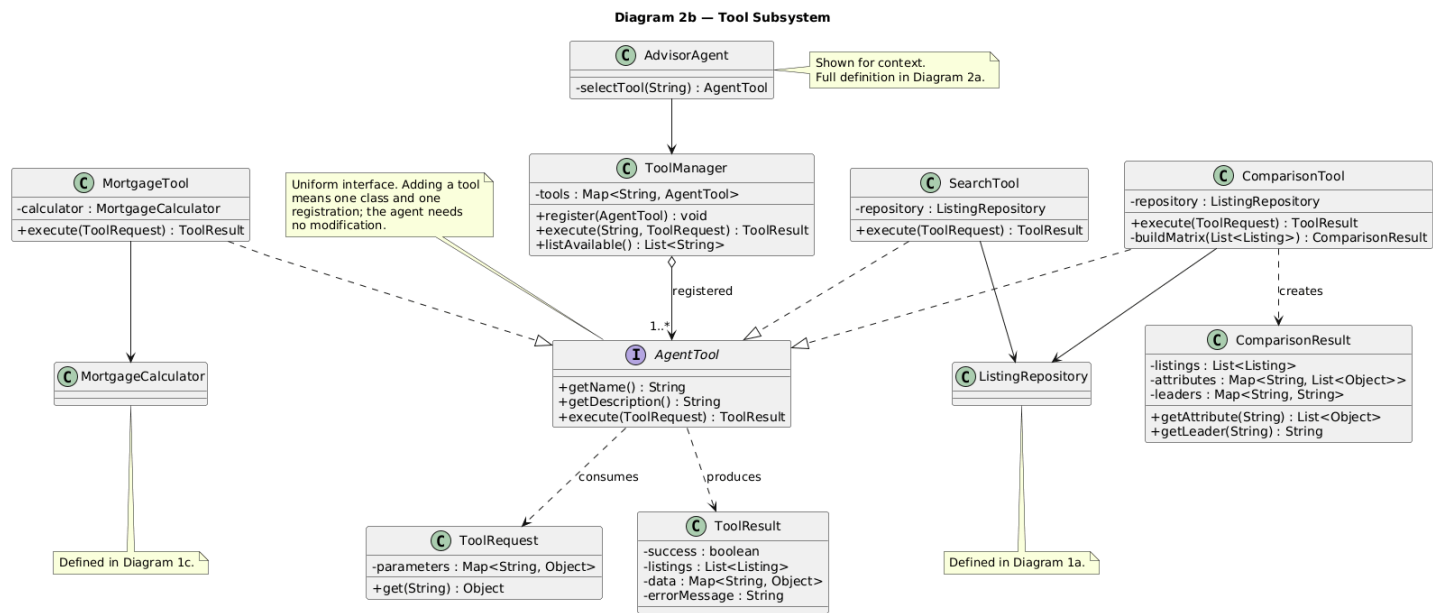


Diagram 2b : Agent Tools



Design Pattern Explanations

Facade - RealEstateService

Problem. The GUI and CLI both need to search, calculate, compare, and manage a shortlist. Without one entry point, each would coordinate the repository, agent, and ranking strategy itself, duplicating that logic.

Participants. RealEstateService is the facade. ListingRepository, AdvisorAgent, MortgageCalculator, and CommandInvoker are the subsystem. MainWindow and CLIRenderer are clients.

Why it fits. It's the only class both frontends touch, so neither can hold business logic; they have no access to the parts needed to implement any.

Without it. Every feature written twice, once per frontend, and the two drift apart.

Strategy - RankingStrategy

Problem. Results sort four ways, switchable at runtime. Conditionals inside the repository would mean editing it for every new ordering.

Participants. RankingStrategy is the interface. The four sorting classes are concrete strategies. RealEstateService holds the active one.

Why it fits. The strategies differ only in their scoring function, which is exactly what Strategy isolates.

Without it. A growing switch statement, and scoring logic tangled with sorting so neither can be tested alone.

Composite - Filter

Problem. Criteria nest arbitrarily: price AND bedrooms AND (condo OR townhouse). A flat list of conditions can't express that.

Participants. Filter is the component. The four field filters are leaves. AndFilter and OrFilter are composites.

Why it fits. ListingRepository.findMatching calls matches on one Filter, never knowing whether it's a single predicate or a deep tree.

Without it. A repository method signature listing every possible criterion, growing with each new one.

Command - ShortlistCommand

Problem. Shortlist actions need undo, which requires each action to know how to reverse itself.

Participants. ShortlistCommand is the interface. AddToShortlist and RemoveFromShortlist are concrete commands. CommandInvoker holds history. Shortlist is the receiver.

Why it fits. The invoker pops commands off a stack and reverses them without knowing what any of them does.

Without it. The shortlist would maintain its own snapshot history, mixing state management into an unrelated class.

Factory Method - ListingParserFactory

Problem. Listings load from JSON or CSV, decided by the file at runtime.

Participants. ListingParser is the product. JsonListingParser and CsvListingParser are concrete products. ListingParserFactory is the creator.

Why it fits. The import routine asks for a parser and calls parse, never naming a concrete class. A third format costs one class.

Without it. An extension check repeated at every call site, coupling import to every parser.

Observer- SearchObserver

Problem. Both frontends must update when results change, but the service shouldn't know which are attached.

Participants. SearchObserver is the interface. MainWindow and CLIRenderer observe. RealEstateService is the subject.

Why it fits. The service depends only on the interface, so frontends can be added or removed freely.

Without it. The service holding direct references to both UIs, reversing the dependency direction and making it untestable without a UI.

Adapter - LLMClient

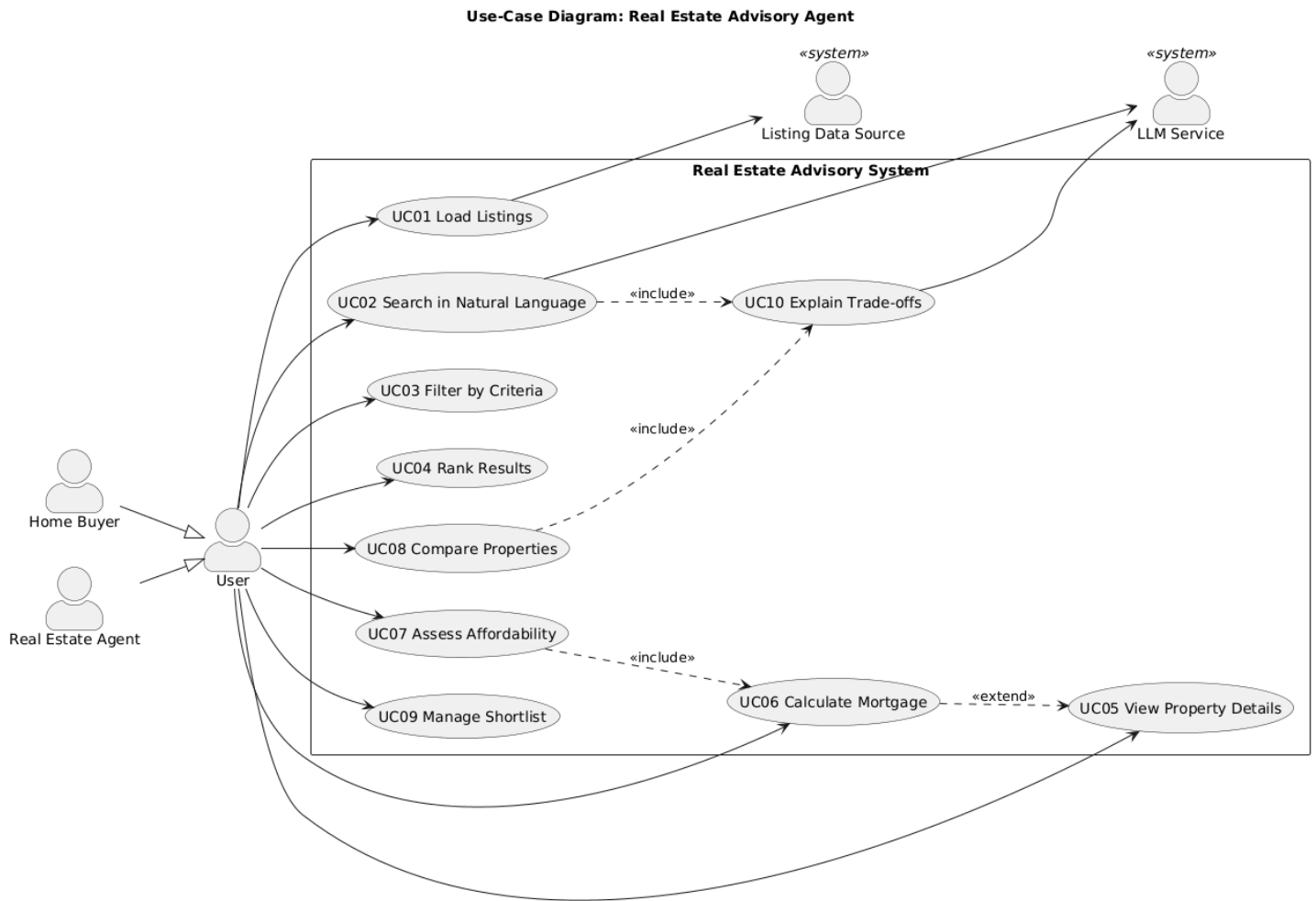
Problem. Calling the language model API directly would make every agent test hit the network and return non-deterministic results.

Participants. LLMClient is the target interface. AnthropicLLMClient adapts the real API. MockLLMClient returns fixed responses. AdvisorAgent and RequirementExtractor are clients.

Why it fits. The interface is shaped around what the application needs rather than the provider's API. MockLLMClient makes the agent layer unit-testable offline, which is what separates the deterministic and behavioral test suites in Stage 3.

Without it. Every agent test becomes a slow, flaky integration test, and switching providers means editing every call site.

Use-case Diagram



Use-Case Descriptions

UC01: Load Listings

- **Actors:** User (primary), Listing Data Source (secondary)
- **Goal:** Populate the system with property records.
- **Preconditions:** A listing file exists at the configured path.
- **Trigger:** Application startup, or File > Load Dataset.
- **Main success scenario:**
 - System locates the data file.
 - System selects a parser based on file format.
 - System parses and validates each record.
 - System stores valid listings in the repository.
 - System displays a load summary.
- **Alternative flows:**
 - 1a. File not found: system reports the expected path and stops.
 - 2a. Unsupported format: system reports the supported formats.
 - 3a. Malformed record: system skips it, logs the reason, and continues.
- **Postconditions:** Repository contains all valid listings.

- **Related features:** F01

UC02: Search in Natural Language

- **Actors:** User (primary), LLM Service (secondary)
- **Goal:** Find properties by describing requirements in plain language.
- **Preconditions:** Listings are loaded.
- **Trigger:** User submits a message in the chat panel or CLI ask.
- **Main success scenario:**
 - User enters a free-text request.
 - System sends the text to the LLM for criteria extraction.
 - System merges extracted criteria with the session criteria.
 - System filters and ranks listings.
 - System displays results and the updated criteria panel.
 - System includes UC10 to explain the results.
- **Alternative flows:**
 - 2a. Invalid extraction: system retries once, then asks the user to rephrase.
 - 2b. LLM unavailable: system tells the user to use the criteria panel instead.
 - 4a. No matches: system identifies the most restrictive criterion.
- **Postconditions:** Session criteria updated; results displayed.
- **Related features:** F02, F10

UC03: Filter by Criteria

- **Actors:** User
- **Goal:** Narrow results using explicit form controls.
- **Preconditions:** Listings are loaded.
- **Trigger:** User changes a criteria control or runs CLI search.
- **Main success scenario:**
 - User adjusts a criterion.
 - System updates the session criteria.
 - System builds a filter and evaluates it against all listings.
 - System displays matching results.
- **Alternative flows:**
 - 1a. Minimum price above maximum: system flags the error and does not search.
- **Postconditions:** Results reflect current criteria.
- **Related features:** F03

UC04: Rank Results

- **Actors:** User
- **Goal:** Order results by a chosen priority.
- **Preconditions:** A result set is displayed.
- **Trigger:** User selects a ranking mode.
- **Main success scenario:**
 - User selects a ranking mode.
 - System applies the matching strategy.
 - System redisplay results in the new order.

- **Alternative flows:**
 - 2a. Listing lacks the ranked attribute: system places it last.
- **Postconditions:** Results shown in ranked order.
- **Related features:** F04

UC05: View Property Details

- **Actors:** User
- **Goal:** See the full record for one property.
- **Preconditions:** The property exists in the repository.
- **Trigger:** User selects a result card or runs CLI show <id>.
- **Main success scenario:**
 - User selects a property.
 - System retrieves the listing and computes derived values.
 - System displays the detail view.
- **Alternative flows:**
 - 2a. Unknown ID: system reports that no such listing exists.
- **Postconditions:** Detail view displayed.
- **Related features:** F05

UC06: Calculate Mortgage

- **Actors:** User
- **Goal:** Know the monthly cost of a property.
- **Preconditions:** A property is selected.
- **Trigger:** User opens the mortgage panel or runs CLI mortgage <id>.
- **Main success scenario:**
 - User enters or accepts default financing terms.
 - System computes payment, total interest, and total cost.
 - System displays the results.
- **Alternative flows:**
 - 1a. Invalid input: system rejects it with a specific message.
 - 2a. Down payment covers the price: system reports no financing needed.
 - 2b. Zero interest rate: system uses the zero-rate formula.
- **Postconditions:** Mortgage figures displayed.
- **Related features:** F06

UC07: Assess Affordability

- **Actors:** User
- **Goal:** Know whether a property is within the user's means.
- **Preconditions:** A property is selected; financial profile entered.
- **Trigger:** User opens the affordability tab.
- **Main success scenario:**
 - System includes UC06 to compute the payment.
 - System adds estimated tax and heating costs.
 - System computes debt service ratios.
 - System displays a verdict and maximum supportable price.

- **Alternative flows:**
 - Pre-1a. No financial profile: system prompts for one.
 - 3a. Zero income: system reports unaffordable.
- **Postconditions:** Verdict displayed.
- **Related features:** F07

UC08: Compare Properties

- **Actors:** User
- **Goal:** Weigh shortlisted properties against each other.
- **Preconditions:** At least two properties are shortlisted.
- **Trigger:** User selects properties and clicks Compare, or runs CLI compare.
- **Main success scenario:**
 - User selects two to four properties.
 - System builds the comparison matrix and marks the leader per attribute.
 - System displays the matrix.
 - System includes UC10 to summarize trade-offs.
- **Alternative flows:**
 - 1a. Fewer than two selected: comparison unavailable.
 - 1b. More than four selected: system caps at four.
 - 4a. LLM unavailable: matrix shown without summary.
- **Postconditions:** Comparison displayed.
- **Related features:** F08, F10

UC09: Manage Shortlist

- **Actors:** User
- **Goal:** Keep and edit a list of properties of interest.
- **Preconditions:** Listings are loaded.
- **Trigger:** User clicks save or remove, or uses the CLI shortlist command.
- **Main success scenario:**
 - User adds or removes a property.
 - System executes the action as a command and records it for undo.
 - System updates the shortlist panel and saves the shortlist.
- **Alternative flows:**
 - 1a. Property already shortlisted: no-op with a notice.
 - 2a. User selects Undo: system reverses the last action.
 - 3a. Save fails: system warns that changes will not persist.
- **Postconditions:** Shortlist updated and persisted.
- **Related features:** F09

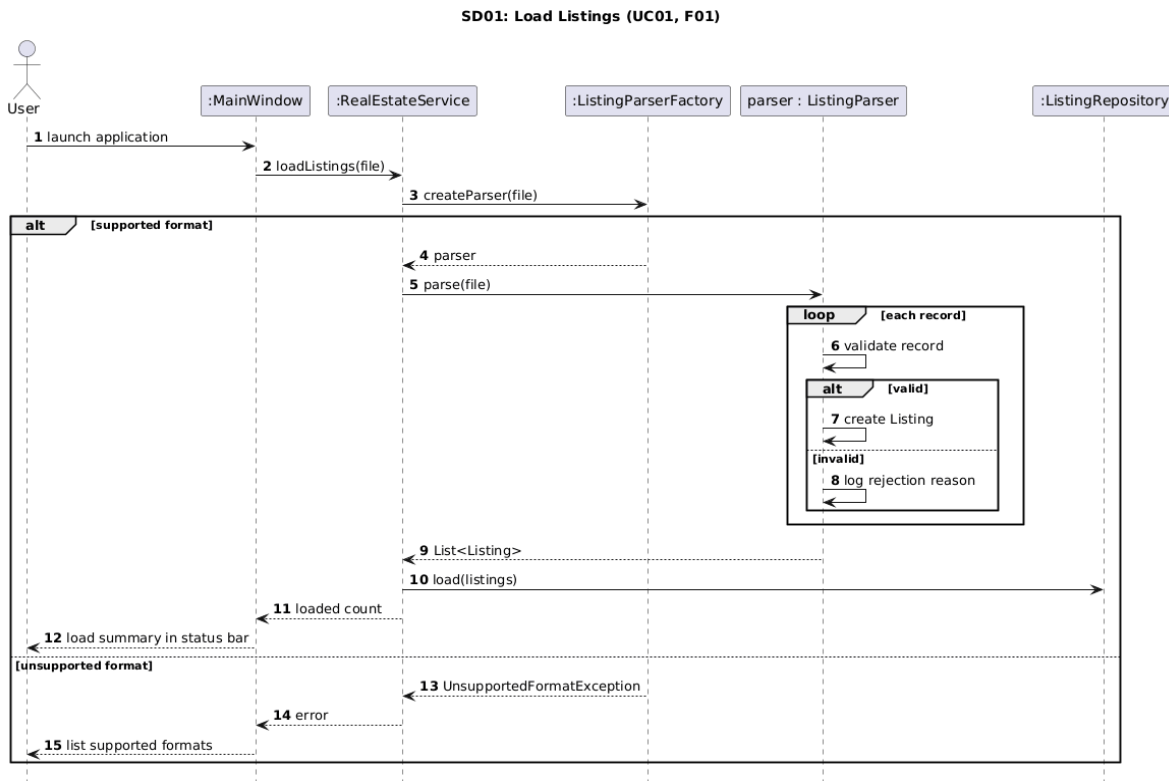
UC10: Explain Trade-offs

- **Actors:** LLM Service (secondary). Not initiated directly by the user.
- **Goal:** Give the user a grounded explanation of their results.
- **Preconditions:** A result set or comparison exists.
- **Trigger:** Included by UC02 or UC08.
- **Main success scenario:**

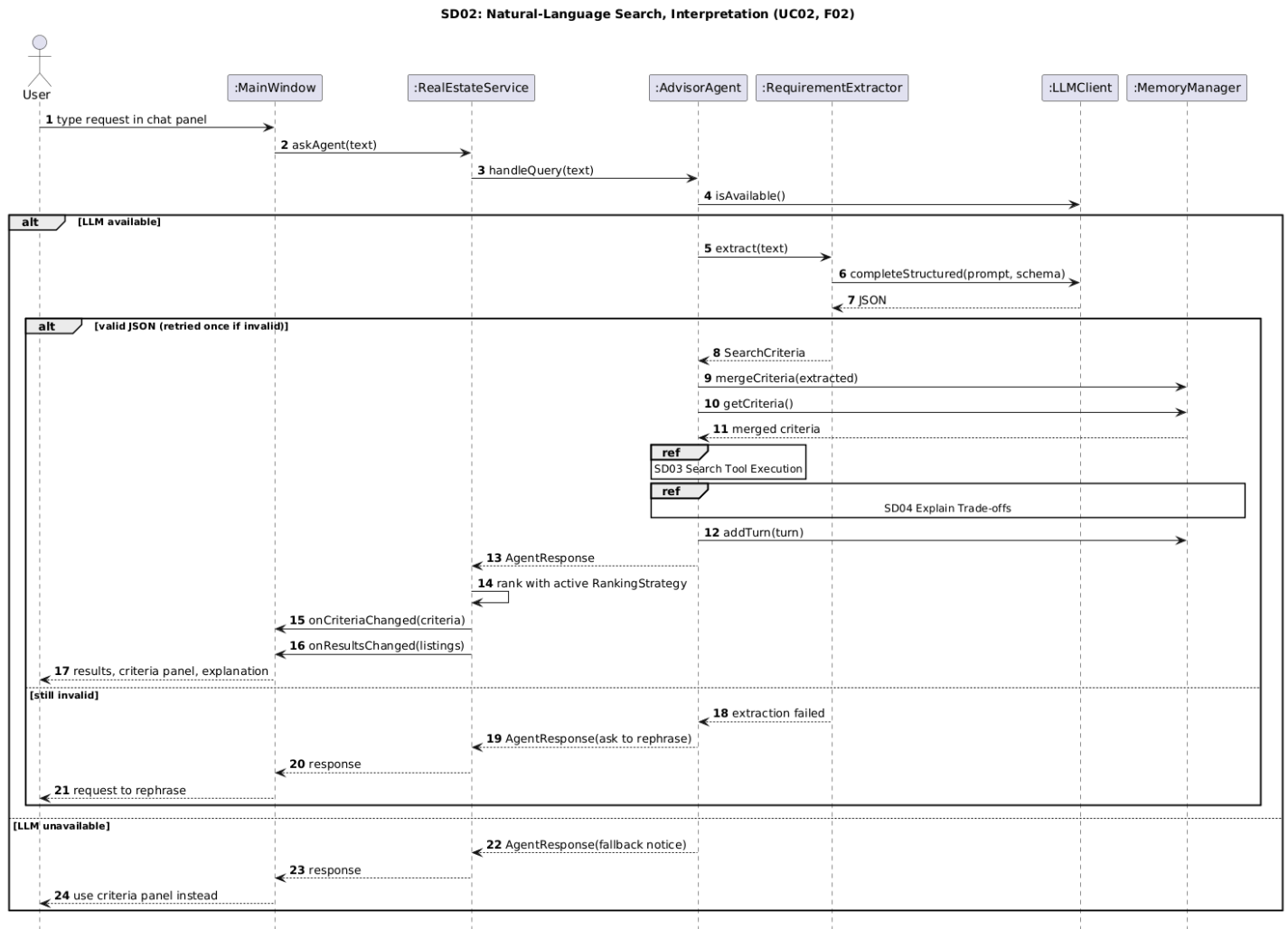
- System builds a prompt with the listings and criteria.
- LLM generates an explanation.
- System checks that every listing mentioned was supplied.
- System displays the explanation.
- **Alternative flows:**
 - 3a. Unsupplied listing mentioned: system regenerates once, then omits the explanation.
 - 2a. LLM unavailable: results shown without explanation.
- **Postconditions:** Explanation displayed, or results shown alone.
- **Related features:** F10

Sequence diagrams

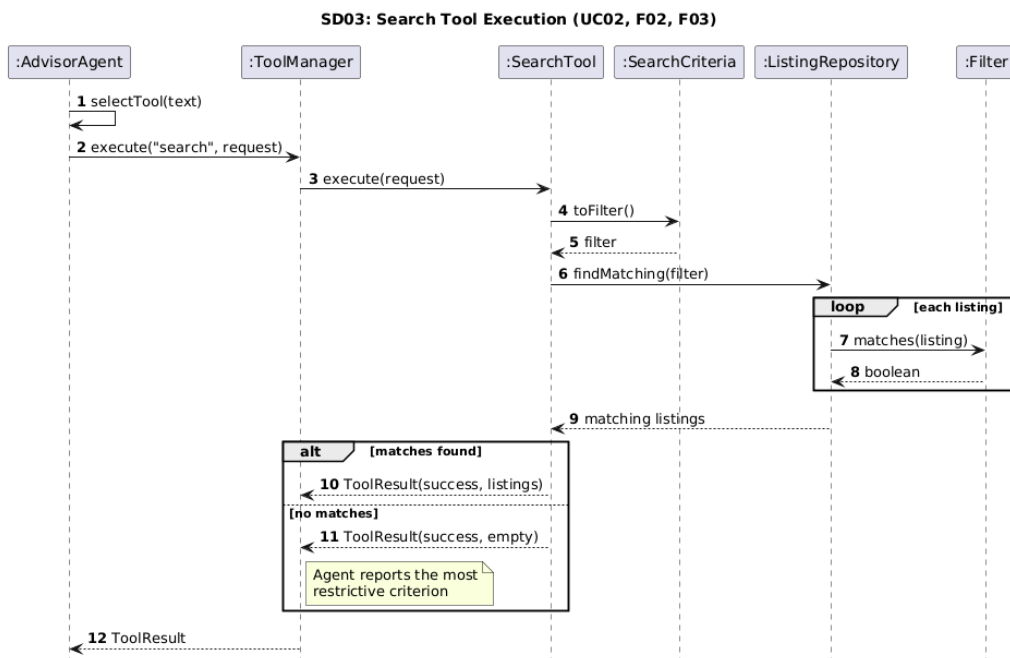
SD01: Load Listings



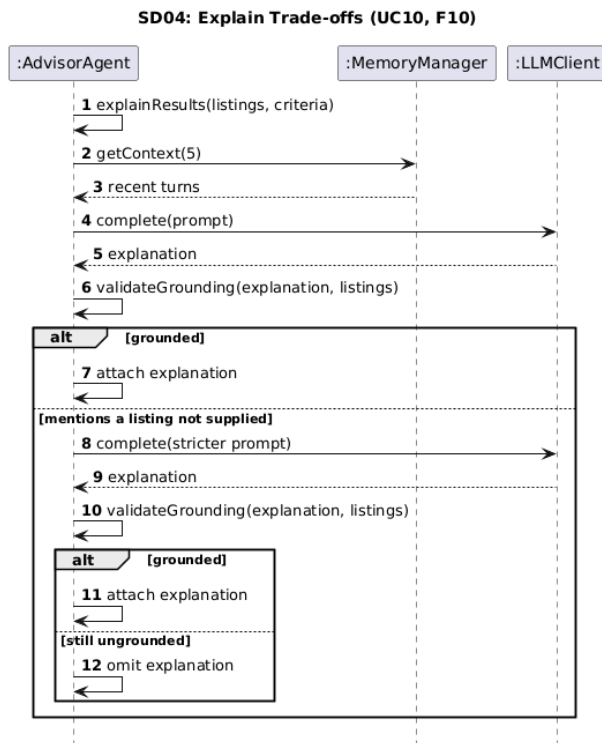
SD02: Natural-Language Search and interpretation



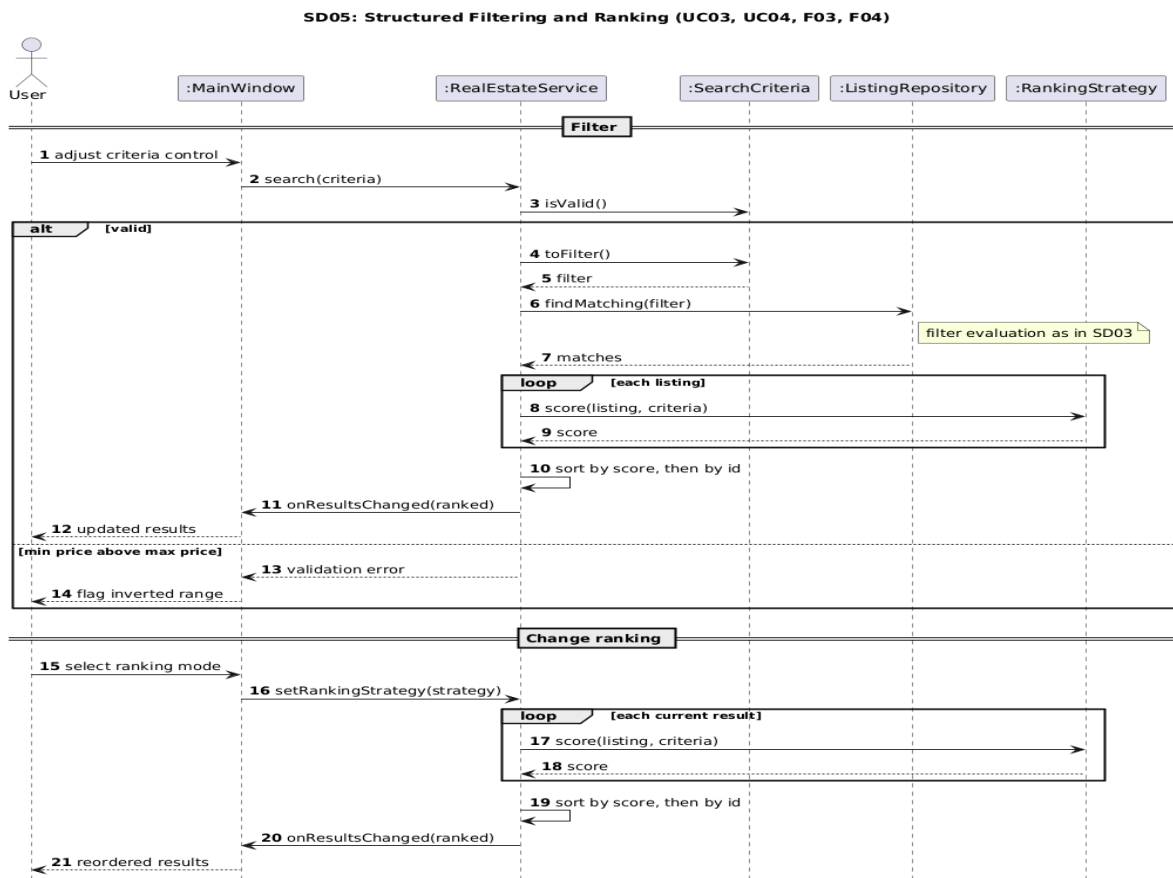
SD03: Search Tool Execution



SD04: Explain Trade-offs

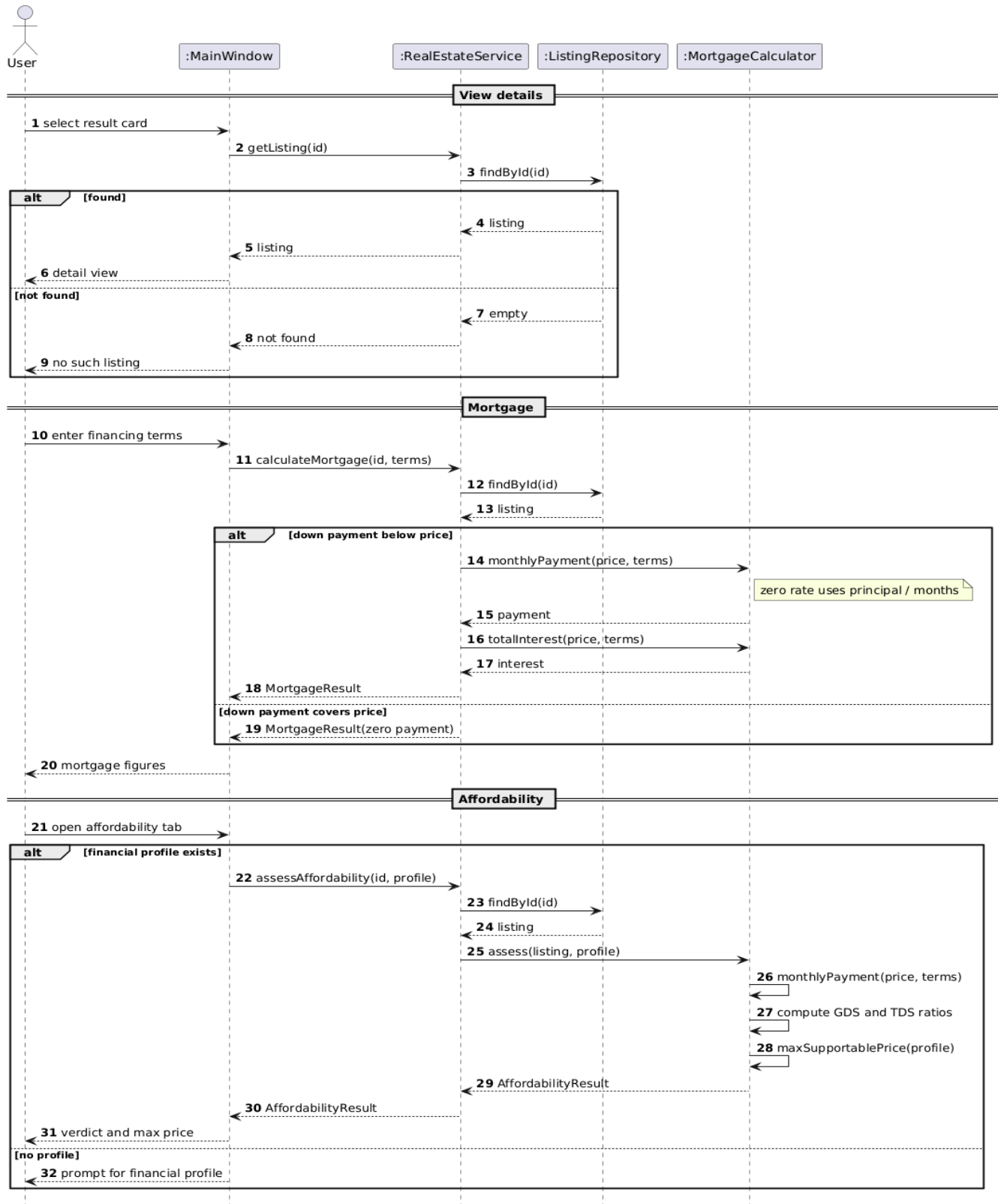


SD05: Structured Filtering and Ranking

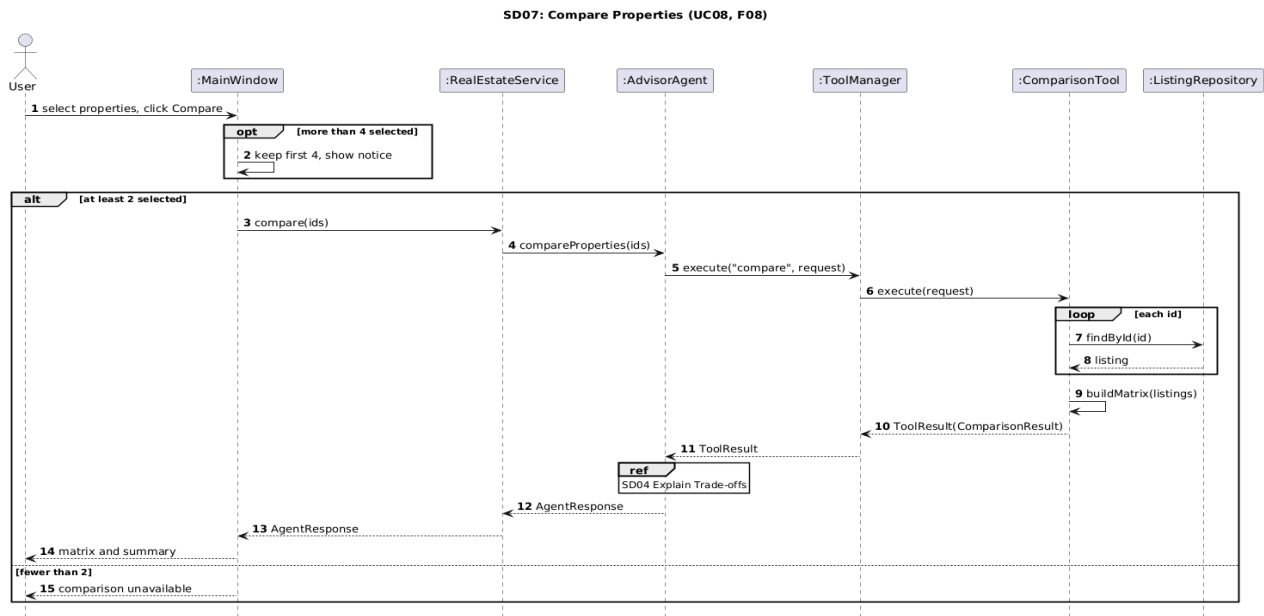


SD06: Details, Mortgage, Affordability

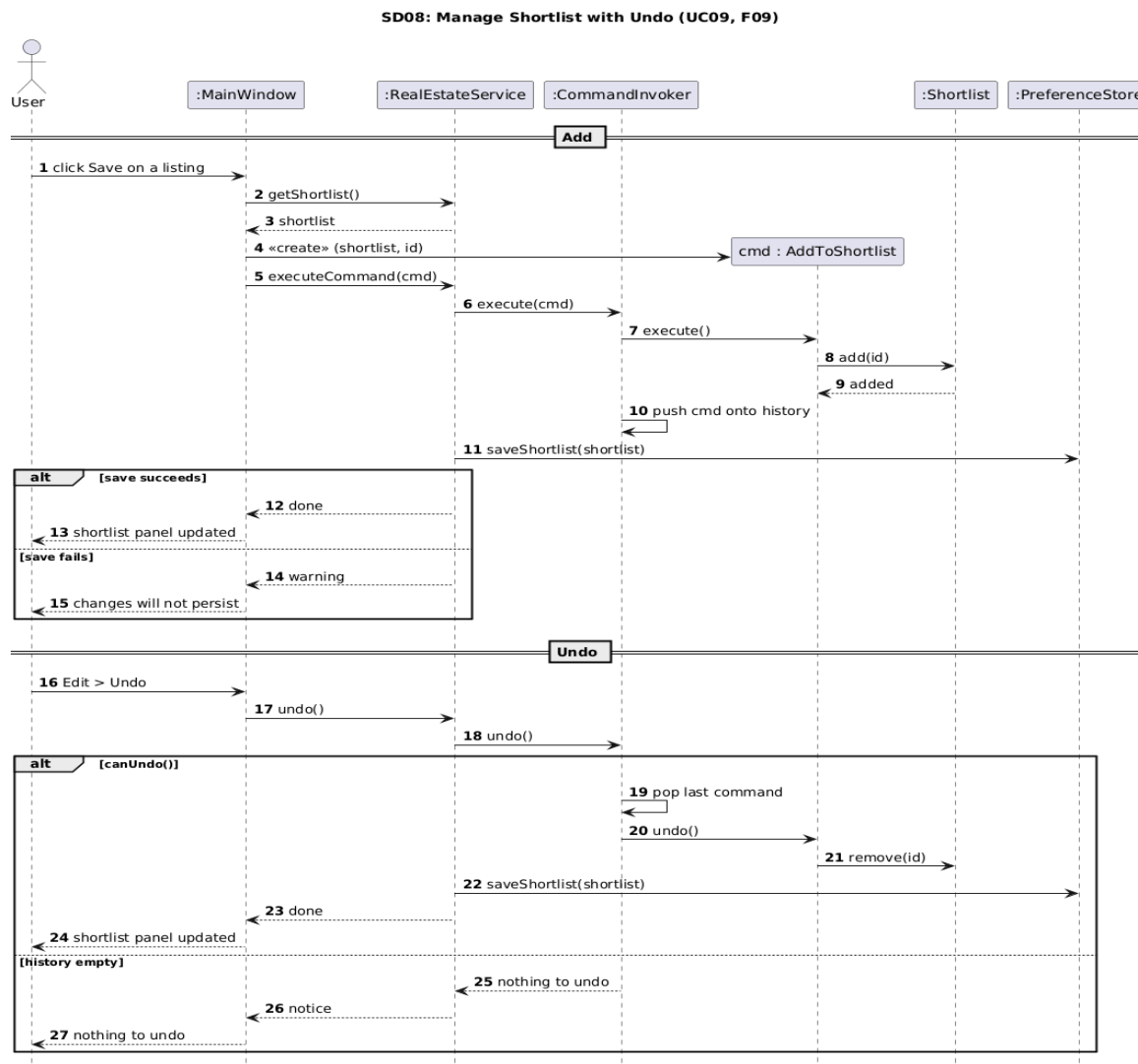
SD06: Details, Mortgage, Affordability (UC05 to UC07, F05 to F07)



SD07: Compare Properties



SD08: Manage Shortlist with Undo



Feature-to-design traceability table

Feature	Description	Type	Use Case	Classes	Key methods	SD	Patterns
F01	Listing data import	Deterministic	UC01	MainWindow, RealEstateService, ListingParserFactory, ListingParser, JsonListingParser, CsvListingParser, ListingRepository, Listing	loadListings(), createParser(), parse(), supports(), load()	SD01	Factory Method, Facade
F02	Natural-language property search	Hybrid	UC02	MainWindow, RealEstateService, AdvisorAgent, RequirementExtractor, LLMClient, MemoryManager, ToolManager, SearchTool, SearchCriteria, ListingRepository	askAgent(), handleQuery(), extract(), completeStructured(), mergeCriteria(), execute(), findMatching()	SD02, SD03	Facade, Adapter, Composite, Observer
F03	Structured criteria filtering	Deterministic	UC03	MainWindow, CLIRenderer, RealEstateService, SearchCriteria, SearchCriteriaBuilder, Filter, AndFilter, OrFilter, PriceFilter, BedroomFilter, TypeFilter, FeatureFilter, ListingRepository	search(), isValid(), toFilter(), matches(), findMatching()	SD05	Composite, Builder, Facade
F04	Result ranking	Deterministic	UC04	MainWindow, RealEstateService, RankingStrategy, PriceAscendingStrategy, PriceDescendingStrategy, ValuePerSqFtStrategy, BestMatchStrategy	setRankingStrategy(), score(), getName()	SD05	Strategy, Observer
F05	Property detail view	Deterministic	UC05	MainWindow, RealEstateService, ListingRepository, Listing	getListing(), findById(), getPricePerSqFt(), getAge()	SD06	Facade
F06	Mortgage payment calculation	Deterministic	UC06	MainWindow, RealEstateService, MortgageCalculator, MortgageTerms, MortgageResult, Listing	calculateMortgage(), monthlyPayment(), totalInterest()	SD06	Facade
F07	Affordability analysis	Deterministic	UC07, UC06	MainWindow, RealEstateService, MortgageCalculator, FinancialProfile, AffordabilityResult, Verdict	assessAffordability(), assess(), maxSupportablePrice()	SD06	Facade
F08	Property comparison	Hybrid	UC08, UC10	MainWindow, RealEstateService, AdvisorAgent, ToolManager, ComparisonTool, ComparisonResult, ListingRepository, LLMClient	compare(), compareProperties(), execute(), buildMatrix(), explainResults()	SD07, SD04	Facade, Adapter
F09	Shortlist management	Deterministic	UC09	MainWindow, RealEstateService, CommandInvoker, ShortlistCommand, AddToShortlist, RemoveFromShortlist, Shortlist, PreferenceStore	executeCommand(), undo(), execute(), canUndo(), saveShortlist()	SD08	Command, Observer, facade
F10	Trade-off explanation	AI	UC10	AdvisorAgent, MemoryManager, LLMClient, AnthropicLLMClient, MockLLMClient, Listing, SearchCriteria	explainResults(), getContext(), complete(), validateGrounding()	SD04	Adapter

Feature explanations

F01 - Listing Data Import

Use case: UC01 · **Sequence diagram:** SD01 · **Patterns:** Factory Method, Facade

Classes involved

- MainWindow - triggers the load at startup and shows the summary.
- RealEstateService - coordinates the load.
- ListingParserFactory - selects a parser for the file format.
- ListingParser, JsonListingParser, CsvListingParser - parse and validate records.
- ListingRepository - stores the resulting listings.
- Listing - the parsed record.

Key methods

RealEstateService.loadListings(), ListingParserFactory.createParser(), ListingParser.parse(), ListingParser.supports(), ListingRepository.load()

Execution

At startup MainWindow calls loadListings(file) on the service, which asks ListingParserFactory for a parser matching the file. The factory inspects the extension and returns either a JSON or CSV parser as a ListingParser; the service never names a concrete type. The parser reads each record, validates required fields and value ranges, and builds a Listing for each valid one. Invalid records are logged with a reason and skipped, so a single malformed row does not abort the import. The resulting list goes to ListingRepository.load(), and the service returns the count for the status bar.

F02 - Natural-Language Property Search

Use case: UC02 · **Sequence diagrams:** SD02, SD03 · **Patterns:** Facade, Adapter, Composite, Observer

Classes involved

- MainWindow - collects the free-text query.
- RealEstateService - the entry point, and the observer subject.
- AdvisorAgent - coordinates extraction, tool selection, and explanation.
- RequirementExtractor - converts language to criteria.
- LLMClient - abstracts the model provider.
- MemoryManager - merges and holds session criteria.
- ToolManager, SearchTool - dispatch and execute the search.
- SearchCriteria, Filter - express the query structurally.
- ListingRepository - evaluates the filter.

Key methods

RealEstateService.askAgent(), AdvisorAgent.handleQuery(), RequirementExtractor.extract(),

LLMClient.completeStructured(), MemoryManager.mergeCriteria(), ToolManager.execute(),
ListingRepository.findMatching()

Execution

The user's text reaches AdvisorAgent.handleQuery() through the service facade. The agent checks LLMClient.isAvailable(), then passes the text to RequirementExtractor, which prompts the model for schema-constrained JSON and parses it into a SearchCriteria. Those criteria are merged into session state by MemoryManager, so a budget stated earlier still applies. The agent selects SearchTool through ToolManager; the tool converts the criteria to a Filter tree and calls ListingRepository.findMatching(), which evaluates the tree against every listing. Results return to the agent, which generates an explanation (F10), and the service ranks them and notifies observers, updating both the results list and the criteria panel.

The division of labour is the point: the model interprets language, and ordinary code does the matching.

F03 - Structured Criteria Filtering

Use case: UC03 · **Sequence diagram:** SD05 · **Patterns:** Composite, Builder, Facade

Classes involved

- MainWindow, CLIRenderer - collect criteria from controls or flags.
- RealEstateService - entry point.
- SearchCriteria, SearchCriteriaBuilder - hold and assemble criteria.
- Filter and implementations - express the query as a tree.
- ListingRepository - evaluates it.

Key methods

RealEstateService.search(), SearchCriteria.isValid(), SearchCriteria.toFilter(), Filter.matches(),
ListingRepository.findMatching()

Execution

Changing a control updates the session SearchCriteria, which the service validates before searching; an inverted price range is caught here. toFilter() builds a composite tree: an AndFilter at the root with a leaf per active criterion, and an OrFilter where a field accepts several values, such as condo or townhouse. The repository calls matches() once on the root and lets the tree recurse, so it needs no knowledge of the query's shape. Results are ranked (F04) and pushed to observers.

This path never touches the language model, which is what makes it exhaustively unit-testable.

F04 - Result Ranking

Use case: UC04 · **Sequence diagram:** SD05 · **Patterns:** Strategy, Observer

Classes involved

- MainWindow - offers the ranking selector.
- RealEstateService - holds the active strategy.
- RankingStrategy and its four implementations - score listings.

Key methods

`RealEstateService.setRankingStrategy()`, `RankingStrategy.score()`, `RankingStrategy.getName()`

Execution

Selecting a ranking mode sets the corresponding `RankingStrategy` on the service. The service calls `score()` for each listing, passing both the listing and the active criteria, since `BestMatchStrategy` needs the criteria to measure fit while the price strategies ignore them. Listings sort by score with a secondary sort on ID, so ties resolve identically on every run - necessary for reproducible tests. Observers then redraw.

Adding a ranking mode means one new class implementing `score()`, with no change to the service.

F05 - Property Detail View

Use case: UC05 · **Sequence diagram:** SD06 · **Pattern:** Facade

Classes involved

- `MainWindow` - renders the detail panel.
- `RealEstateService` - retrieves the listing.
- `ListingRepository` - looks it up by ID.
- `Listing` - computes derived values.

Key methods

`RealEstateService.getListing()`, `ListingRepository.findById()`, `Listing.getPricePerSqFt()`, `Listing.getAge()`

Execution

Selecting a card calls `getListing(id)`, which delegates to `findById()`. That returns an `Optional`, so a missing ID produces an explicit message rather than a blank view. The listing computes its own derived values: price per square foot, and age from year built. Where square footage is absent, `getPricePerSqFt()` signals unavailable rather than dividing by zero. The panel switches to detail mode and renders.

F06 - Mortgage Payment Calculation

Use case: UC06 · **Sequence diagram:** SD06 · **Pattern:** Facade

Classes involved

- `MainWindow` - collects financing terms.
- `RealEstateService` - entry point.
- `MortgageCalculator` - performs the arithmetic.
- `MortgageTerms`, `MortgageResult` - carry input and output.

Key methods

`RealEstateService.calculateMortgage()`, `MortgageCalculator.monthlyPayment()`, `MortgageCalculator.totalInterest()`

Execution

The service retrieves the listing and passes its price with the MortgageTerms to MortgageCalculator. Monthly payment uses the standard amortization formula with the monthly rate derived from the annual one. Two cases bypass it: a zero interest rate, where the formula divides by zero and payment becomes principal over months; and a down payment at or above the price, which returns zero rather than a negative principal. Total interest is total payments minus principal. The MortgageResult returns to the GUI, which recalculates live as terms change.

This class carries no dependency on the agent or the repository, making it the cleanest target in the system for unit testing.

F07 - Affordability Analysis

Use cases: UC07 (includes UC06) · **Sequence diagram:** SD06 · **Pattern:** Facade

Classes involved

- MainWindow - collects the financial profile.
- RealEstateService - entry point.
- MortgageCalculator - computes ratios and the verdict.
- FinancialProfile, AffordabilityResult, Verdict - carry input and output.

Key methods

RealEstateService.assessAffordability(), MortgageCalculator.assess(),
MortgageCalculator.maxSupportablePrice()

Execution

assess() first computes the monthly payment, reusing the F06 path, then adds estimated property tax and heating to produce total housing cost. The gross debt service ratio is housing cost over gross monthly income; the total debt service ratio adds existing obligations. Both are compared against standard thresholds to produce a Verdict of affordable, marginal, or unaffordable. maxSupportablePrice() inverts the calculation at the threshold ratio to tell the user what they can actually carry. Zero income returns unaffordable without dividing, and debt alone exceeding the thresholds is flagged separately, since no property price resolves it.

F08 - Property Comparison

Use cases: UC08 (includes UC10) · **Sequence diagrams:** SD07, SD04 · **Patterns:** Facade, Adapter

Classes involved

- MainWindow - selection and rendering.
- RealEstateService, AdvisorAgent - coordinate.
- ToolManager, ComparisonTool - build the matrix.
- ComparisonResult - carries it.
- ListingRepository - supplies the listings.
- LLMClient - generates the summary.

Key methods

RealEstateService.compare(), AdvisorAgent.compareProperties(), ComparisonTool.execute(),
ComparisonTool.buildMatrix(), AdvisorAgent.explainResults()

Execution

The GUI caps the selection at four and requires at least two before enabling Compare. ComparisonTool retrieves each listing and builds a matrix across price, size, bedrooms, bathrooms, age, price per square foot, and monthly payment, marking the leading property per attribute according to whether higher or lower is better for that attribute. Missing attributes render as unavailable and are excluded from leader determination rather than treated as zero. The completed ComparisonResult passes to the agent, which generates the trade-off narrative through F10.

The split matters: the matrix is deterministic and verifiable, and if the model is unavailable the matrix still renders with only the narrative missing.

F09 - Shortlist Management

Use case: UC09 · **Sequence diagram:** SD08 · **Patterns:** Command, Observer, Facade

Classes involved

- MainWindow - save and remove controls, undo menu item.
- RealEstateService - entry point.
- CommandInvoker - executes commands and holds history.
- ShortlistCommand, AddToShortlist, RemoveFromShortlist - the actions.
- Shortlist - the receiver.
- PreferenceStore - persistence.

Key methods

RealEstateService.executeCommand(), RealEstateService.undo(), ShortlistCommand.execute(), ShortlistCommand.undo(), CommandInvoker.canUndo(), PreferenceStore.saveShortlist()

Execution

A save action creates an AddToShortlist holding the shortlist and the listing ID. CommandInvoker.execute() runs it and pushes it onto the history stack. The command calls Shortlist.add(), which returns false for a duplicate, making the repeat a no-op rather than a second entry. Observers update the panel and count badge, and the shortlist persists; a persistence failure warns the user while leaving in-memory state correct.

Undo pops the last command and calls its undo(), which reverses the specific action. The invoker never knows what it is undoing, which is what lets further undoable actions be added later without touching it.

F10 - Trade-off Explanation

Use case: UC10 · **Sequence diagram:** SD04 · **Pattern:** Adapter

Classes involved

- AdvisorAgent - builds the prompt and validates the response.
- MemoryManager - supplies conversation context.
- LLMClient, AnthropicLLMClient, MockLLMClient - generate the text.
- Listing, SearchCriteria - the grounding data.

Key methods

AdvisorAgent.explainResults(), MemoryManager.getContext(), LLMClient.complete(),
AdvisorAgent.validateGrounding()

Execution

After a search or comparison, the agent assembles a prompt containing the returned listings, the active criteria, and recent conversation turns from MemoryManager. The prompt restricts the model to the supplied properties. The response passes through validateGrounding(), which checks that every listing ID mentioned appears in the supplied set. A failure triggers one regeneration with a stricter prompt; a second failure drops the explanation and renders results alone.

This validation step is the system's main defence against fabricated recommendations, and it is the primary target of the behavioral tests in Stage 3. When the model is unavailable, results display without narrative rather than failing.